Basic Terminology

1. Scalar:- Single quantities
   → constants, variables

2. Vector:- ordered list of scalar values, called attributes

$$a = \begin{bmatrix} 11 \\ 12 \\ 10 \end{bmatrix}$$

3. Draw a matrix X.

Feature 1 → Size (sq. ft)
Feature 2 → No. of bedroom
Feature 3 → Age of home (years)

$$X = \begin{bmatrix} 1000 & 3 & 5 \\ 850 & 2 & 1 \\ 1200 & 4 & 4 \\ 550 & 1 & 3 \end{bmatrix}$$

3. Sets:- { } → unordered collection of unique elements.

[a, b] → includes a, b
(a, b) → excludes a & b.
{0, 1} → just 2 elements
[0, 1] → ∞
(0, 1) → ∞

* Membership:- when an element x belongs to S
  $x \in S$.

* Intersection:- new set with common elements.

* Union

P.T. → $|S_1 \cup S_2| + |S_1 \cap S_2| = |S_1| + |S_2|$

LHS ⇒ $|S_1| + |S_2| - |S_1 \cap S_2| + |S_1 \cap S_2|$ ∵ $A \cup B = A + B - A \cap B$

$|S_1 + S_2| = |S_1 + S_2|$

LHS = RHS

1] Partition the set as
→ $S_1 \setminus S_2$ → Elements only in S_1
$S_2 \setminus S_1$ or $S_2 - S_1$ → Elements only in S_2
$S_1 \cap S_2$ → Elements common in both S_1 & S_2
The 3 sets above are disjoint

$$|S_1 \cup S_2| = |S_1 \setminus S_2| + |S_2 \setminus S_1| + |S_1 \cap S_2|$$

w.r.t

$$|S_1| = |S_1 \setminus S_2| + |S_1 \cap S_2| \quad \text{—— (I)}$$

$$|S_2| = |S_2 \setminus S_1| + |S_1 \cap S_2| \quad \text{—— (II)}$$

from (I) & (II)

$$|S_1| + |S_2| = |S_1 \setminus S_2| + 2|S_1 \cap S_2| + |S_2 \setminus S_1|$$

$$|S_1| + |S_2| = |S_1 \setminus S_2| + |S_1 \cap S_2| + |S_1 \cap S_2| + |S_2 \setminus S_1|$$

$$\boxed{|S_1| + |S_2| = |S_1 \cup S_2| + |S_1 \cap S_2|}$$

$$|S_1 \cup S_2| = |S_1| + |S_2| - |S_1 \cap S_2|$$

→ Prev. proof: LHS = RHS.

→ This comes in Dataset overlap.

Data leakage ⟶ same data in training & testing data
↳ results are inaccurate

* Data is ~~relevant~~ when $A \cap B$ is 0.

Model Prediction

$$|S_1 \cup S_2| + |S_1 \cap S_2| = |S_1| + |S_2|$$

union ⟶ coverage

S_1 ⟶ samples predicted +ve by model A ⟶ agreement

S_2 ⟶ samples predicted +ve by model B

Then

$S_1 \cap S_2$ ⟶ High confidence prediction

$S_1 \cup S_2$ ⟶ all +ve predicted by at least 1 model

This logic is used in:-

1] Ensemble method

2] Consensus-based filtering

3] Weak prediction.

2] Binary classification ⟶ [Email spam]

Total email = 10

Task:- classify each email as spam or not spam

step 1 ⟶ Define the sets

S_1 ⟶ Email predicted as spam by model A

S_2 ⟶ Email predicted as spam by model B.

These are just email id

$S_1 = \{e_1, e_2, e_4, e_7\}$

$S_2 = \{e_2, e_3, e_4, e_8\}$

step 2:- Intersection (Agreement)

$|S_1 \cap S_2| = \{e_2, e_4\}$.

* When combining predictions from multiple models, common predictions must be counted carefully or else we overestimate model accuracy.

Places where we see this proof:-

* Capital Sigma Notation

$$\sum_{x_i=1}^{n} = x_1 + x_2 + x_3 \ldots x_n$$

Euclidean norm $||x||$ ⟶ size of length

given by $\sqrt{\sum_{i=1}^{n} x(i)^2}$

The distance between vector a & b is given by

1] MSE $= \frac{1}{n} \sum_{i=1}^{n} (y - \hat{y})^2$

Total error across dataset. we want to minimize this.

2. Gradient Descent:-

$$\frac{\partial L}{\partial w} = \sum_{i=1}^{n} x_i (\hat{y}_i - y)$$

used in model training

3. Linear Regression

$$\hat{y} = \sum w_j x_j$$

weighted feature sums.

4] Naive Bayes classifier

$$P_y = \sum_x P(Y/x) P(x).$$

5] classification Accuracy

$$d = \frac{1}{n} \sum_{i=1}^{n} 1 (\hat{y} - y).$$

Returns → 1 / 0 counts correct prediction

6] L2 Regularization

$$\lambda \sum_{j=1}^{d} w_j^2$$

Penalizes large weight

7] NN [Loss + Backpropagation]

$$L = \sum_{i=1}^{n} loss_p.$$

21/1/26

→ What is ML?

→ Types of ML :- classified into 4 types :-
- Supervised
- Unsupervised
- Reinforcement
- Semi-supervised.

Supervised learning

1 Dataset Representation :-

$$X = \begin{bmatrix} x_{11} \dots x_{1m} \\ x_{21} \dots \\ \dots \\ x_{n1} \dots x_{nm} \end{bmatrix} \quad y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

2 Learning objective

$f : x \rightarrow y$

$\hat{y} = f(x)$

Model

$\hat{y} = w^T x + b$

Loss Function : $\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y})^2$

Binary Classification (Ans).

Features :-
- Avg order value last 30 days
- Time series since last purchase
- Discount usage
- App session count
- Customer support interaction

Learning Objective

$f(x) = P(\text{churn})$

↳ False Positive.

y : 0 - customer stayed active
1 :- customer churned in 60 days

Model :- Logistic Regression / Gradient Boosted trees

Loss function :- Binary Cross Entropy.

Goal is to minimize Churn misclassification.

Problem 1:- An e-commerce company notices that customers who previously made large, frequent purchases suddenly become inactive without complaints or churn signals. The business want to identify at-risk customer early so retention offers can be triggered proactively.

a] Define the input features & target variables
b] Specify the dataset representation
c] Identify a suitable model
d] Write a learning objective in words
e] Write the loss function
f] Write the Traing goal.

Problem 2:- Failure Risk Analysis :- Regression

A factory sees machine passing quality checks but failing soon after deployment, causing warranty claims. Management wants to predict early-life failure risk before shipping.

Features:- 1. Session Sensor Reading.
2. Batch ID
3. Assembly Line
4. Duration of test.

Learning Objective:-
Predict y [Time to failure]
y → predict # days to first failure

Model:- Any Regression Models
↳ Random forest
↳ Linear Regression

Loss function:- MSE.

Problem 3:- A bank observes that customers with acceptable credit scores still default, especially under changing economic conditions. The goal is to improve default prediction beyond credit score:-

Goal:- Reduce False Negative
Predicted safe but customer defaulted.

Learning Objective:-
y → 1:- defaulted
0:- Fully repaid.

Model:- classification Problem.

Loss function:- Binary class Entropy.

Features:- salaried
↳ credit score
→ :

Problem 4:- A SaaS company sees user actively using the product during trial but not converting to paid plans. The business wants to predict conversion likelihood and intervene with targeted nudge.

Problem-5 :- A logistic firm meets SLA targets on average, yet some route fail unpredictably, causing customer dissatisfaction. They want to predict delivery delay duration before dispatch.

Goal:- Minimize delay prediction Error.

Feature:- Dispatch
Weather
Time of dispatch.
Driver experience
Warehouse congestion

22/1/26

Classification vs Regression

Algo	Type	Main Library	Importing Lib / class
1. Linear Regression	Regression	scikit-learn	from sklearn.linear_model import LinearRegression
2. Logistic Regression	Classification	scikit-learn	from sklearn.linear_model import LogisticRegression
3. K-NN	Classification / Regression	scikit-learn	from sklearn.neighbors import KNeighborsClassifier / KNeighborsRegressor
4. Decision Tree	Classification / Regression	scikit-learn	from sklearn.tree import DecisionTreeClassifier / DecisionTreeRegressor
5. Random Forest	Classification / Regression	scikit-learn	from sklearn.ensemble import RandomForestClassifier / RandomForestRegression
6. SVM	classification / Regression	scikit-learn	from sklearn.svm import SVC / SVR

Algo	Type	Main Library	Importing Lib.
Naive Bayes	classification	scikit-learn	from sklearn.naive_bayes import GaussianNB
Neural Network (MLP)	classification/ Regression	scikit-learn	from sklearn.neural_network import MLPClassifier/ MLPRegressor
XG Boost	classification/ Regression	scikit XG Boost	from xgboost import XGBClassifier/ XGBRegressor
Deep Neural Network	C / R	TensorFlow/ PyTorch	import tensorflow as tf import torch.

Accuracy

out of all prediction, how many were correct?

Formula :- $\frac{TP+TN}{TP+TN+FP+FN}$

used when classes are balanced & cant use when not balanced.

Precision

when model says Yes, how often is it correct?

Formula :- $\frac{TP}{TP+FP}$

We dont want to miss tve case

what it controls :- FP.

when to use → actions are expensive
false alarm are costly

Example → Retention offers
Fraud investigation.

Sensitivity (Recall)

out of all actual Yes, how many did we catch?
↳ correctly identified

Recall :- $\frac{TP}{TP+FN}$ →

what it controls → FN

when to use ⟶ when positive case is costly

F1-Score used for imbalance dataset
↳ Single score that balances precision & recall

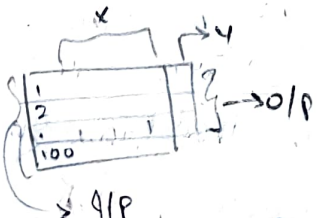
Formula :- $\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

Specificity :- out of all actual -ve, how many did we correctly reject $\frac{TN}{TN+FP}$.

Support
↳ No formula [its a coward]
↳ How many actual samples belong to each classes

Confusion Matrix :-

Actual \ Predicted	+ve	-ve
+ve	TP	FN
-ve	FP.	TN



Steps in all libraries

1] Define the model
2] Provide labelled data (X, y) → I/P
3] Choose a loss function (explicitly or implicitly)
4] Train the model
5. Evaluate prediction

Q] E-commerce company wants to predict whether a customer will churn in the next 60 days so that retention action will be taken.

Target (label)

$y = \begin{cases} 1 & \text{customer churns} \\ 0 & \text{customer stays.} \end{cases}$

Write a code to implement this by logistic reg. & xgboost with app. libraries

Data → Customer_churn.csv

Logistic

⟹ import ~~from sklearn~~ import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

df = pd.read_csv(customer churn)

X = data.drop("churn", axis=1)

y = data["churn"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

lr = LogisticRegression(max_iter=1000)

lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)

XGBoost [Gradient Boosted Trees] → Ensemble method

import pandas as pd

from sklearn.model_selection import train_test_split

from XGboost import XGClassifier

df = pd.read_csv("customer churn")

X = data.drop("churn", axis=1)

y = data["churn"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

xgb = XGBClassifier(n_estimators=100, max_depth=4, learning_rate=0.1, eval_metric="logloss", random_state=42)

xgb.fit(X_train, y_train)

y_pred_xgb = xgb.predict(X_test).

o/p of logistic reg:-

Accuracy:- 0.82 → 82% of customers were correctly classified

	precision	recall	f1-score	support
0	0.85	0.88	0.68	320
1	0.75	0.68	0.71	180
accuracy			0.82	500
macro avg	0.80	0.78	0.79	500
weight avg	0.82	0.82	0.82	500

For class 0

Precision:- % Among all the model predicted as "not churned" 85% truly did not churn.

Recall:- 0.88
↳ of all that model predicted as "Not churn" 88% was right (correctly identified)

For class 1

Precision = 0.75
↳ Among all, the model predicted as "churn" 75% truly churned. correct.

Recall:- 0.68
↳ It catches 68% of actual churner

∴ The model is conservative → avoid false alarm but misses some at-risk customers.

Here Recall is imp as high recall = few misses
high precision = few false alarm

o/p of XG Boost

Accuracy = 0.88

	precision	recall	f1-score	support
0	0.90	0.92	0.91	320
1	0.85	0.85	0.83	180

Costly mistake → customer who churn but model fail to identify.

Recall of churn helps with that.

Log. Regression recall = 0.68
→ misses 32% churner.

XG Boost recall = 0.82
→ misses only 18% of churner

Losing money → Recall

Precision → what ever money spent is spent on correct people i.e.
→ Retention offer costs money.
XG boost → catches more churners.
wastes fewer offers.

Types of learning:-

Unsupervised
→ label free
→ identify missing patterns
→ detect outliers.

unsupervised → only unlabelled data
→ find hidden pattern or grouping
→ self guided looks for similarities
→ algo:- K-Mean; PCA; Autoencoder.

Semi-Supervised → Mix of labelled + unlabelled dataset
→ uses small labelled set to guide larger unlabelled sets
→ combines "teacher" & "solo" learning.
→ medical imaging; label propagation → algo.

Indirect feedback

Reinforcement Learning

→ No predefined data; learns from Environment
→ Maximize long-term Rewards via actions
→ Direct feedback from environment.
  → Learns from Experience
    → Robotics, Game AI,
      Q learning, SARSA, etc.

29/1/26

Unsupervised Learning

An e-commerce company has customer data containing age, income & spending behaviour. The company wants to segment customer into meaningful groups for marketing.

1) Discover groups in data ... [K-means]

2) A city traffic dept. collects large amt of GPS location data from vehicles moving across the city over several days. Each record represents the position of vehicle at a given time, and the dataset contains only GPS co-ordinates with no labels such as road names, etc.

→ objective:- automatically identify traffic hotspot & busy road segments

  → Density Based Clustering → DBSCAN.

2] Dimensionality Reduction

A medical dataset contains 300 diagnostic features collected from patient tests. There are ....

PCA, AutoEncoders.

→ A supermarket stores transaction records of items purchased together. There are no labels between... identify frequently co-occuring pattern.

↳ FP Growth, Apriori

3] Discover Hidden Pattern / Association.

→ An online learning platform...
The platform wants to discover hidden relationship
↳ ?

4] DETECT OUTLIERS

↳ Isolation Forest, SVM.

Goal	Algo	Importing Library / Class	Input
1] Discover Groups / Cluster	K-Means	from sklearn.cluster import KMean	Numeric feature matrix X = [[age, income ...]] Output Cluster labels for each data points (0, 1, 1, 2 ...)
	DBSCAN	from sklearn.cluster import DBSCAN	
2] Reduce Dimensionality	PCA	from sklearn.decomposition	
	t-SNE		

3.	Discover Pattern / Association	Apriori	from mixtend.frequent_pattern import apriori	Transaction matrix (binary) milk, bread.. o/p:- Frequent itemsets rules (Support Confidence)
4.	Detect Anamolies / Outlier	Isolation Forest	from sklearn.ensemble import IsolationForest	Numeric feature matrix o/p:- normal anomaly labels (1=normal)
		One-class SVM	from sklearn.svm import OneClassSVM	
5.	Learn Representation	Autoencoders	import tensorflow as tf / import torch	

model.transduction → returns an array of points with some labels assigned

Semi Supervised learning works only if-

1] Cluster Assumption → Points in the same cluster have same label

2] Smoothness Assumption

3] Manifold Assumption

Algorithms of Semi Supervised.

Pseudo labeling

1] Train classifier on labeled dataset.

Self - Training

→ only takes high confidence predicts. Add those emails to the labeled set
Retrain & repeat
eg images.

Label Propagation

↳ Groups on similarity, generates graph based on that labeling is done
eg emails
↳ some words belong to same label.

Auto encoder + classifier.

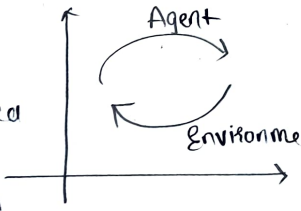
↳ denoising, &
→ compressing data.
→ dimensionality reduction.

Reinforcement Learning

↳ agent lives within environment.

agent → goal
↳ rewarded

agent - goal
↳ penalised



The agent perceives the state of its environment as a vector of features

Iterative Process :- Agent learns by executing actions, observing results.

Sequential Decision :- goal is long term rather than immediate. RL is characterised by sequential decision-making.

eg] Self-Driving Car:-

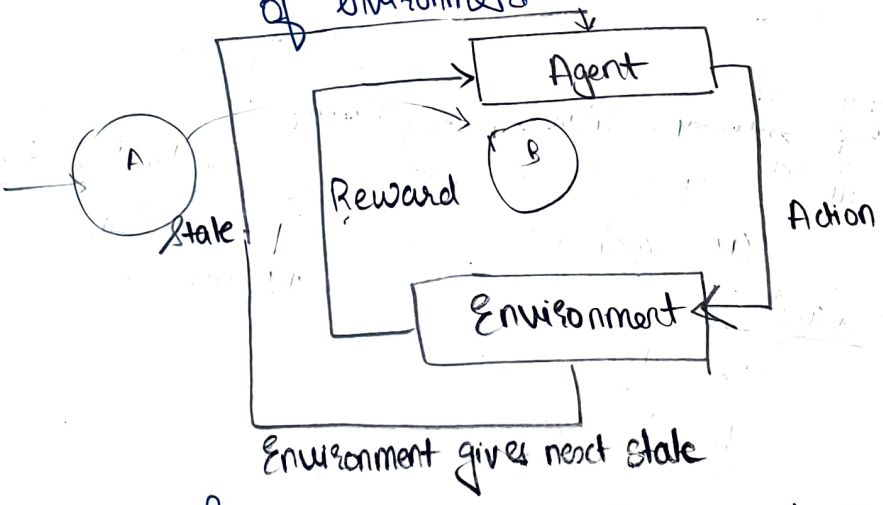
Core Mechanics:- State, Action & Reward

State (s): representation of current situation of the environment, provided to the agent as input.

Action (a) agent executes specific action in non-terminal states to influence the environment.

Reward (r):- Different actions yield different numericals which can be +ve or -ve.

State Transition :- Executing an action moves the machine from its current state to a new state of environment.



Optimal Policy :- Primary goal to learn a function that takes the feature vector of a state as i/p & o/p the optimal action to execute in that state.

Maximising Reward:- action is defined as optimal if it maximizes the expected average long-term reward.

Exploration vs Exploitation:- Similar to multi-armed Bandit problem, RL system must balance exploring the environment to find new reward with exploiting new reward.

Applications

1] Navigating physical spaces → vaccum clear
Robotics →

2] Strategic Games Playing master games with long term strategy depth

3] Autonomous System → Navigation, obstacle avoidance

4] Logistic & Resource Management:-
→ when there are complex supply chains

Types

1] Value-Based

2] Policy-Based

3] Model-Based

4] Hybrid Approach:- Actor-Critic Method
→ value + policy + model

Case study:- AWS DeepRacer

Agent & Environment:- In DeepRacer ecosystem, the car serves as agent, and track represents the environment.

Reward Function:- Goal:- Encourage the car

Initialize Environment
[Track, car pos.]
↓
Observe current state
(pos, speed, wheels)
↓
Select action
(steering, throttle)
using current policy
↓
Execute Action
Move car on Track

Call Reward Function
reward = f(params)
↓
update policy
[RL]
↓
Episode finished
↓ Yes / No
Yes → Reset car
No → Next time step

Multi-armed Bandit problem: [MABP]

def Hypothetical solution where MABP models a situation where an agent must choose among multiple action (called arms) repeatedly:-

where
- → Each arm gives a random reward
- → Reward probabilities are unknown
- → Goal:- Maximize total reward over time

eg] used for customer selection group.
Each customer → slot
maybe they would select or not

core characteristics -No state, Decision is made Repeatedly.

Arm	Action	Reward
Arm	Show Ad A	click / No click
Arm 1	Show Ad B	Click / No click
Arm 2	Show Ad C	Click / No click
Arm 3		

In context of applied ML
the resources = user of system.
choices = various model
reward = metric [user engagement time / click through rate]

A/B testing [long term goals use statistical technique]

work on statistical experiment where:-
users are split into fixed groups
Each group receives a different variant
Performance is measured after sufficient data
One option is chosen at the end.

Group	variant	
Grp 1	old website	After enough user → Compare conversion rate → Choose the better variant.
Grp 2	new website	

Use A/B Testing when:-

Changes are low freq.
Statistical confidence is required
Environment is stable

eg] Major UX or product redesign
→ Care about long term metrics

Use Multi-arm Bandit when

Decisions are repeated
User interaction is ongoing
You want to reduce regret in real time

→ Add or content selection
→ Email subject line optimization

Example:- A/B Testing

eg] Uber used A/B Testing across core business logic

what they tested?
→ Surge price algo.
→ Driver incentive program.

Why?
→ They affect revenue, fairness and user trust.
→ Decision must be carefully validated before full rollout

Example:- Udemy [Multi Arm Bandit]

Built a real-time bandit system for course recommendations
Millions of courses
Need to rank each user

MABT for ranking & recommendation.
Adjusts which courses are shown based on performance

Why?
→ user preference change constantly
→ Need fast adaption, not just post experiment analysis

Data used directly & Indirectly:-

Data used in ML project is categorized

Data used directly

↳ consists of fundamental basics of dataset.
Every entity is transformed into individual

eg] NER

Data used Indirectly:- some ↳ common gazetteer ↳ pre compiled dict.

Raw AND Tidy Data

↳ must undergo feature Engineering to be into usable state. [eg Image, PDF, Audio].

Tidy → necessary but not sufficient condition. [eg → csv file]
↳ consistent column positioning

Training AND Holdout Set

Three - way split:

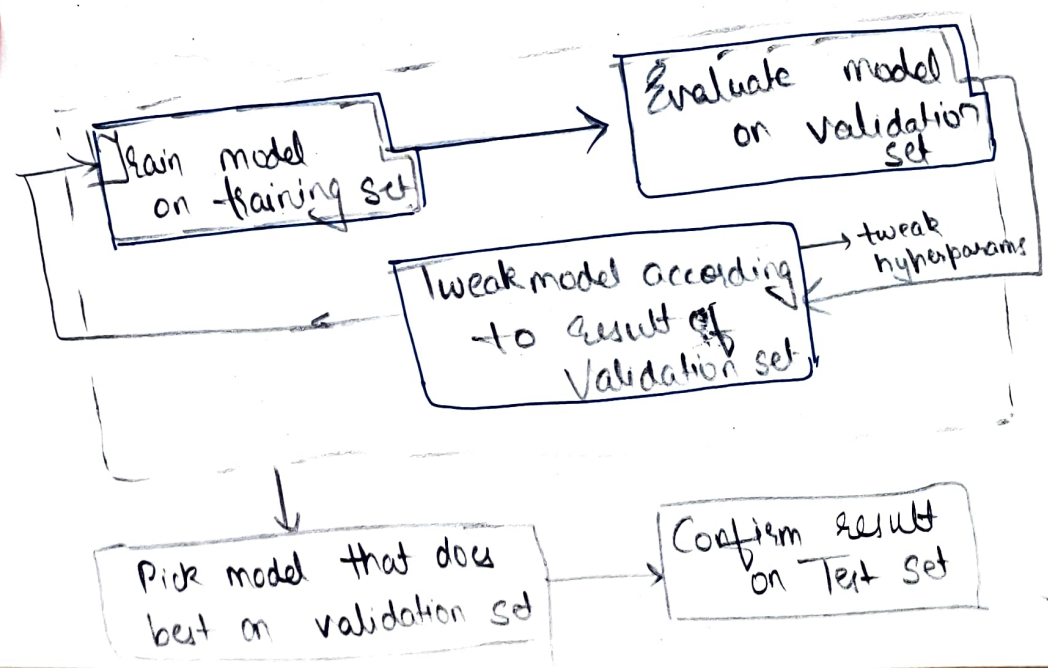
Train → largest, used to build stat. model.
↳ model learn the task.

Validate → Intermediate
↳ which model is good?

Test → How good is the model?

we change parameters based on validate.

↳ Holdout sets as model have never seen it



Validation Set utility

↳ used to compare diff. algo or diff config of same algo.

Primary tool for hyperparameter tuning

eg] Linear & Logistic Regression

→

Test Set & Generalization

eg] SMOTE

↳ used for final performance reporting

↳ used to

→ success is measured by how well model generalizes prediction for unseen data, indicating its ability to generalize

→ validation & test set should follow same statistical distribution.

Baseline

→ Simple algo or heuristic that provides a reference point for model comparison

→ Establishing baseline gives analyst confidence that a more sophisticated machine learning solution actually provides value.

Common Baseline Algo [Basic Baseline Approach]

1] Random Prediction → Randomly chooses a label based on distribution of training set.
checks whether learned
50% spam 50% not spam

2] Zero Rule → in classification, it predicts the most freq. class.
in reg. it predicts sample average since 70% that class

3] Simple Linear Model → uses basic linear or logistic reg for other ML model.

* Advanced Baseline Approach:-
1] Existing rule-based system:- simple heuristic can serve as powerful baseline
→ doesn't use ML but simple if-else ladder

Recommends → similar articles as last read.
→ Ranks most recent or most popular

Simple rule often performs well & must be beaten by ML model to justify their complexity.

2] Human Performance → If human can perform a task, their accuracy serves gold standard baseline.
Avg. human accuracy = 90%.
⇒ If model achieves 70% accuracy, its not acceptable as human can do better.

↳ Part of Human performance.

3] Crowdsourcing :- eg Amazon Mechanical Turk can be used to gather ensemble of human prediction to establish accuracy.

eg Task → Identify offensive content on Insta.
Sol → Each post sent to 5 people
→ Find labels

Amazon Mechanical Turk [MTurk] crowdsourcing platform where small tasks are completed by human workers

Parameters vs Hyperparameters

↳ Internal variable
during training model tries to learn optimal value.
eg] weight & bias.

Hyperparameter → data cant learn this
↳ external input to algo or pipeline
eg] max. of depth of tree, No. of k in knn

Tweaking this allows to manage critical trade off :- eg bias vs variance.

* Parameters are optimized on Training set,
Hyperparams are evaluated on Validation set

Choose a hyperparam → Train the model on training set learn param → Evaluate on validation set → change hyperparams

Select hyperp. that gives best result

Model Based vs Instance-Based learning.

Model Based:- We train the model on data → og data discarded
- we dont need the data anymore
- o/p of training model = statistical model.
  - learning weights & bias.

These algo generally seek to find a ~~best~~ hypersurface or plane / line that separate class
eg] SVM, Linear or logistic Reg.

Instance-Based → uses entire dataset as model, rather than learning a set of fixed parameter.
- slower than Model Based.

o/p is label that appears more freq. among
eg] K-NN

Deep Learning vs Shallow learning → less feature req; learns directly from parameters.

very imp to know why this prediction happen use shallow. → bit more explainable than deep.

In Deep:- learning happens through prev. layer [input or hidden] & not from raw features
eg] CNN [has layered arch]

---

use shallow learning when → when we can pinpoint the metrics
- → hence cost of error is high use shallow

→ dataset is small
→ feature are well engineered
→ interpretability is important.

---

use deep learning when:-
→ large amt of data is available
→ Raw or unstructured data
→ F.E is difficult
→ Accuracy is required.

- → CNN
- → Speech recognition
- → Machine translation
- → Chatbot & LLM

Transfer Learning

1] first few layer is freezed & last few are unfreezed as no. of classes is different

eg ImageNet initial dataset

Imagenet → Randomly initialized weights → N/W trained to classified 1000 classes → Fine tune more weights update → New data & class.

Training vs Scoring

Model Training Phase

called as inference → applying a prev. trained model to new i/p to generate prediction.

Scoring logic → handles errors
↳ done at run time where queries from production user are transformed into feature vector & passed through model.

how well is model handling business needs so after deployment

→ time is acceptable → throughput || holistic

operational monitoring during ~~train~~ scoring focuses on latency.

when to use ML

1] when coding is complex → too many rules & exceptions, partial but useful sol is accepted

2] prdo. constantly changing → i/p has correlated attributes → Benefits from inference.

→ Rule based system require constant data distribution changes updates.

fraud detection

No pattern recog → No ML

eg] webscraping, fraud detection freq

3] Perceptive Problem → tasks involves human like perception.
I/P are high dimensional, & cant be transferred to rule
eg → Image recognition & Speech recognition.

4] Phenomenon not studied → exploratory studies
→ no clear scientific or mathematical model
eg] Personalized med; System & N/w logs & Predicting human behaviour
social media apps.

5] when problem has simple objectives.
eg → Spam vs not Spam classification problem

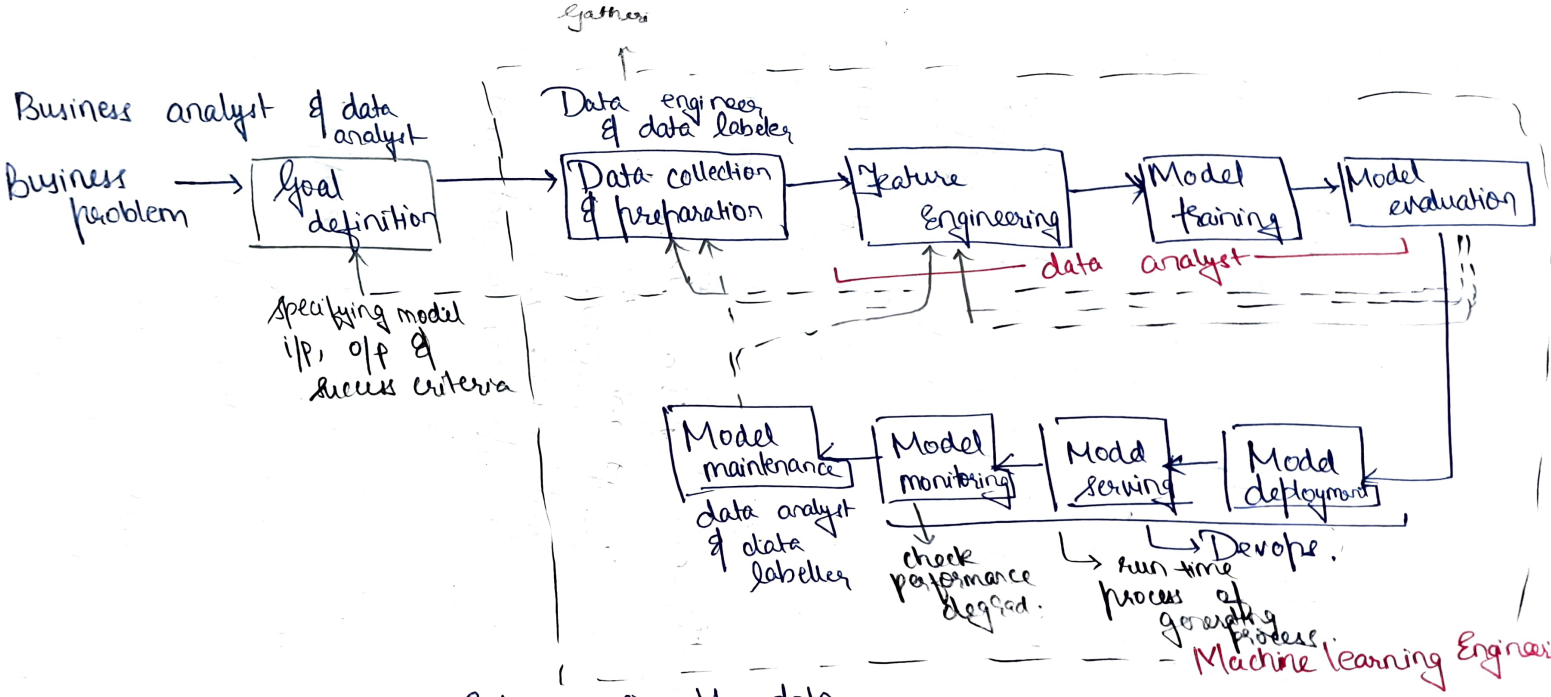
Not useful when:-
task requiring interacting decisions.

6] When its cost effective

Not useful :-
→ when cost of error too high:-
→ med. cases; aircraft
→ deployment should be fast. & to market as fast as possible
→ getting right data is complex
→ could be solved using simple trad. software.
→ simple heuristic is enough.

Machine Learning Engineering → Applied ML
application of scientific principle, tools & techniques from both ML & SE to design & build complex computing systems.

When ML research often focuses on inventing new general purpose algo; MLE is essentially Applied Machine Learning.



Data collection & prep → Gathering ↳ sizeable, reusable, reliable data

F.E = Transforming raw data into num. vectors.

Model traing → apply algo & tuning hyperparameter

MLOps process of automating ML using Devops methodologies such as CI & CD (continuous delivery)

↳ Keeping Brain alive.

MLE → application of scientific principle, tools & technique from both ML + SE to design & build complex computing system.

↳ Build the brain

→ Model design & selection

→ Feature engineering

→ Training & eval.

→ Tuning ML ideas into working software.

MLOps :-

→ Automation of ML pipeline

→ Model Deployment

→ Monitoring performance & drifts

→ Scaling & reliability.

Organizing ML Projects :-